🔁 Python Loops (for and while)

Let Python repeat the boring stuff while you take the credit!

Python | Beginner | Ages 9–16

InfiBee Coders

💡 The Big Idea

- A loop tells Python to repeat some code again and again.
- Without loops, you would copy and paste the same lines many times.
- Python has two main loops: `for` and `while`.
- Each time the loop runs the code once, we call it an iteration.
- Loops save time and make programs shorter.

🔁 The for Loop

- Use a for loop when you know how many times to repeat, or when you have a collection to go through.
- Syntax: `for variable in something:`
- The variable takes a new value on each iteration.
- The code to repeat must be indented by 4 spaces.
- You can loop over a list, a string, or a range of numbers.

🔢 The range() Function

- `range(5)` gives 0, 1, 2, 3, 4. It starts at 0 and stops BEFORE 5.
- `range(1, 6)` gives 1, 2, 3, 4, 5. The start is included, the stop is not.
- `range(0, 10, 2)` gives 0, 2, 4, 6, 8. The third number is the step.
- A negative step counts down: `range(5, 0, -1)` gives 5, 4, 3, 2, 1.
- `range()` is the best friend of the for loop.

⏳ The while Loop

- Use a while loop when you do not know how many repeats you need.
- Syntax: `while condition:`
- Python checks the condition before every iteration.
- The loop keeps going while the condition is True and stops when it becomes False.
- Something inside the loop must change the condition, or it never ends!

🎛️ Controlling a Loop

- `break` stops the loop right away.
- `continue` skips the rest of this iteration and jumps to the next one.

- A loop can sit inside another loop. This is called a nested loop.
- In a nested loop, the inner loop finishes fully for each step of the outer loop.

💻 Code Examples

for loop with range — python

```python
# Print numbers 1 to 5
for number in range(1, 6):
    print(number)
```

▶ **Output:**

```
1
2
3
4
5
```

This loop prints the numbers 1 to 5. The stop value 6 is not included.

Looping over a list — python

```python
fruits = ["apple", "banana", "mango"]

# fruit holds one item on each iteration
for fruit in fruits:
    print("I like", fruit)
```

▶ **Output:**

```
I like apple
I like banana
I like mango
```

The loop visits each fruit in the list one at a time.

while loop countdown — python

```python
count = 3

while count > 0:
    print(count)
    count = count - 1  # change the condition variable

print("Blast off!")
```

▶ **Output:**

```
3
2
1
Blast off!
```

The loop runs while count is bigger than 0. Each time, count goes down by 1 so the loop can end.

break and continue python

```python
for number in range(1, 10):
    if number == 3:
        continue  # skip 3
    if number == 5:
        break  # stop at 5
    print(number)
```

▶ Output:

```
1
2
4
```

continue skips the number 3. break stops the loop when the number reaches 5.

A nested loop: times table python

```python
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end="  ")
    print()
```

▶ Output:

```
1  2  3
2  4  6
3  6  9
```

The inner loop runs completely for every row.

Adding up with a loop python

```python
total = 0
for number in range(1, 11):
    total = total + number
print("Total:", total)
```

▶ Output:

```
Total: 55
```

total starts at 0 and grows by each number.

📚 Key Terms

Loop
Code that repeats until it is told to stop.

Iteration
One single run through the loop's code.

for loop
A loop that goes through items or a set number of times.

while loop
A loop that repeats as long as a condition is True.

range()
A function that makes a sequence of numbers for a loop.

Loop variable
The variable in a for loop that holds the current item.

Indentation
The spaces at the start of a line that show it is inside the loop.

break
A keyword that exits the loop immediately.

continue
A keyword that skips to the next iteration.

Infinite loop
A loop that never stops because its condition never becomes False.

🧮 Formulas & Rules

```
for item in collection:
for i in range(stop):
for i in range(start, stop):
for i in range(start, stop, step):
```

```
while condition:
break     # leave the loop
continue  # skip to next iteration
range(n) makes n numbers: 0 to n-1
```

✏️ Worked Examples

Q: Print the numbers 1 to 5 using a for loop.

A: Step 1: range(1, 6) gives 1 to 5 because the stop number is not included. Step 2: Write for number in range(1, 6): and print(number) inside. Output: 1 2 3 4 5, each on its own line.

Q: Count down from 3 to 1 with a while loop, then print Blast off!

A: Step 1: Set count = 3. Step 2: while count > 0: print count, then do count = count - 1. Step 3: When count reaches 0 the condition is False and the loop stops. Step 4: Print Blast off! after the loop.

Q: Add up the numbers 1 to 10.

A: Step 1: Make total = 0 before the loop. Step 2: for number in range(1, 11): add number to total. Step 3: After the loop, print total. The answer is 55.

🚫 Common Mistakes

- Forgetting the colon (:) at the end of the for or while line.
- Forgetting to indent the code inside the loop, which causes an IndentationError.
- Thinking range(5) includes 5. It stops at 4.
- Forgetting to change the variable in a while loop, which makes an infinite loop.
- Putting code that should run once (like the final print) inside the loop by indenting it too much.

👨‍👩‍👧 Parent Corner

- Ask your child to explain the difference between for and while using a daily routine, like brushing each tooth versus stirring until the sugar dissolves.
- Play 'Human Loop': call out a number and have your child do that many jumps. Then ask what the Python code would look like.
- Let your child predict the output of a short loop on paper before running it. Then check together.
- If a program freezes, stay calm. Show how to stop it (Ctrl+C) and ask which line should have changed the condition.
- Ask them to make a times table for a favourite number using a loop, and celebrate when it works.

🎯 Quick Check

1. What numbers does range(4) produce?

✅ 0, 1, 2, 3.

2. Which loop would you choose to repeat until the user types 'quit'?

✅ A while loop, because we do not know how many times it will repeat.

3. What does break do?

✅ It stops the loop immediately.

4. What does continue do?

✅ It skips the rest of the current iteration and goes to the next one.

5. Why does while True: print('Hi') never stop?

✅ The condition True never becomes False, and there is no break, so it is an infinite loop.